

13. Сортировка

Чтобы воспользоваться эффективными алгоритмами поиска, данные должны быть упорядочены в том или ином порядке, иначе говоря, отсортированы.

Сортировка (или упорядочение), также как и поиск, является одной из часто используемых при обработке данных операций и применяется ко многим структурам данных: массивам, связным спискам, деревьям и графам.

Существуют свидетельства, что первой программой для ЭВМ с хранимой программой (с архитектурой Джона фон Неймана) была именно программа сортировки [8].

13.1. Классификация алгоритмов сортировки

Количество алгоритмов сортировки достаточно велико. В первом издании книги *Дональда Кнута «Искусство программирования для ЭВМ»* [8] упоминаются о существовании (на момент написания книги) около двадцати пяти алгоритмов сортировки. В настоящее время это число ещё больше увеличилось, и вместе с экзотическими и мало распространёнными алгоритмами приближается к 40 (и даже уже превышает 40, а с модификациями ещё больше – около 42-44, скорее всего и эти цифры заниженные). Все эти алгоритмы можно классифицировать по нескольким признакам.

1. В зависимости от того, сортируются данные в оперативной памяти машины (ОЗУ) или во внешнем ЗУ, сортировка бывает *внутренней* или *внешней*.

2. В зависимости от того, какая структура данных подвергается упорядочению, может быть сортировка массива, связного списка, дерева (пирамиды), графа.

3. В зависимости от того, как структура данных меняется в процессе сортировки может быть *сортировка списка* (если элементы структуры данных перемещаются), или *сортировка таблицы адресов* (если элементы не перемещаются, а сортируется имеющаяся или создаваемая таблица адресов). В таблицу адресов могут быть добавлены ключи, тогда получается *сортировка таблицы ключей*.

4. По особенностям функциональной реализации алгоритмы сортировки делятся на группы:

Первичные:

- сортировка перестановками (обменная);
- сортировка выбором (отбором);
- сортировка вставками;

Вторичные:

- сортировка подсчетом;
- сортировка слиянием;
- распределяющая сортировка.

5. По широте применения – универсальные (большинство перечисленных выше групп алгоритмов) и алгоритмы для конкретных случаев, не универсальные.

6. По признаку перестановки совпадающих данных в процессе сортировки – алгоритмы, не меняющие порядок следования совпадающих ключей (иногда такие алгоритмы не совсем удачно называют «устойчивыми» или «стабильными»), например, **сортировка вставками**, и меняющие порядок следования («неустойчивые», «нестабильные»).

7. По характеру зависимости времени работы от размера N структуры данных:

– $O(N^2)$ – N -квадратичные (это самые простые алгоритмы из первых трёх функциональных групп);

– $O(N^k)$, где $1 < k < 2$ (например, $k = 1,6667$ или $k = 1,5$ или $k = 1,27$ и т.п. – это **алгоритм Шелла**); возможен также случай, когда $k = 3$ – это один из самых неудачных и неэффективных алгоритмов, который получил название **глупая** (или **дурацкая**) **сортировка**;

– $O(\log_2 N)$ – логарифмические (например, **быстрая сортировка**, сортировка **слиянием** и **пирамидальная сортировка**), более точное выражение – $O(N \log_2 N)$;

– $O(N)$ – алгоритмы, требующие линейного времени, например, **«блинная» сортировка**.

Возможны также некоторые другие принципы разделения алгоритмов сортировки, в том числе по занимаемой дополнительной памяти.

Большинство алгоритмов имеет модификации. В книге Дональда Кнута упоминаются более двадцати алгоритмов внутренней сортировки и их комбинации.

При таком большом разнообразии важными оказываются **критерии выбора алгоритма**:

- средняя скорость сортировки (среднее время, удельная скорость);
- скорость в лучшем и худшем случаях (или минимальное и максимальное время);
- естественность поведения;
- переставляются ли элементы с совпадающими значениями (ключами).

Скорость (или время) сортировки определяется количеством операций сравнения и перестановки. У разных алгоритмов время работы находится в экспоненциальной или логарифмической зависимости от числа элементов структуры данных (массива) N .

Лучший случай – это ситуация, когда структура данных уже отсортирована в нужном порядке, **худший случай** – структура данных отсортирована в порядке, обратном требуемому. Время в лучшем и худшем случаях важны, если возможно частое повторение одной из этих ситуаций. Часто при хорошей средней скорости алгоритмы имеют неприемлемую скорость в худшем случае.

Естественность поведения означает, что если массив уже упорядочен, должно выполняться минимальное число операций (в идеале 0). Максимальное число операций выполняется, если массив отсортирован в обратном порядке.

Кроме перечисленных критериев при **выборе алгоритма** следует принимать во внимание:

- размер структуры данных, которую предстоит сортировать (некоторые алгоритмы, высокоэффективные в средних случаях, например **быстрая сортировка**, не показывают своей высокой эффективности для массивов малого размера);
- степень исходной упорядоченности сортируемых данных;
- требования к необходимой памяти (разным алгоритмам может требоваться $O(1)$, $O(N)$ или $O(N^2)$ памяти машины). Известно несколько примеров, когда эффективные алгоритмы тре-

буют таких больших объемов машинной памяти, что это сводит на нет все преимущества «эффективности» алгоритма;

– трудоёмкость реализации алгоритма в сравнении со сложностью решаемой в программе задачи (например, **пирамидальная сортировка** является одним из самых сложных алгоритмов сортировки). Эффективные, но сложные алгоритмы могут быть нежелательными, если готовые программы будут поддерживать люди, не участвовавшие в разработке этих программ. Необходимо предусмотреть возможность того, что эффективные, но достаточно сложные алгоритмы не будут востребованы из-за своей сложности и трудоёмкости их изучения и освоения.

13.2. Пузырьковая сортировка

Самый простой алгоритм группы **обменных** сортировок (или сортировок **перестановками**) получил название **пузырьковой** сортировки. Этот алгоритм считается самым простым из универсальных, одновременно являясь одним из самых малоэффективных (т.е. медленных). Пузырьковая сортировка заключается в **сравнении** соседних элементов и, при необходимости, в их **перестановке**. При неубывающем упорядочении элементы «всплывают» как пузырьки каждый до своего уровня (а другие элементы одновременно «тонут»).

Кроме пузырьковой сортировки к группе **обменных** относятся также **быстрая** сортировка и сортировка **«расчёской»**. Следует отметить, что операция перестановки используется не только в алгоритмах обменной сортировки, но и во многих других.

Описание алгоритма:

Используются **два цикла**. **Внешний** проходится $N - 1$ раз, это гарантирует, что даже в худшем случае каждый элемент будет находиться на своем месте. В этом цикле устанавливается (смещается) граница между отсортированной и неотсортированной частями структуры данных. Во **внутреннем** цикле выполняются непосредственно операции сравнения и перестановки. Перестановка элементов производится методом **«трёх вёдер»**: содержимое первого элемента помещается в буферную переменную, на его место помещается содержимое второго элемента, на

место которого помещается из буфера «старое» содержимое первого элемента. Количество проходов **внутреннего** цикла в каждом следующем проходе **внешнего** цикла уменьшается (от $N - 1$ до 1). Считается, что в среднем число проходов **внутреннего** цикла равно $N/2$. Процесс сортировки носит «треугольный» характер (рисунок 13.1).

Рассмотрим исходный массив символов, содержащий значения f, d, a, c, b, e.

В процессе работы программы массив будет изменяться следующим образом (показаны исходное состояние массива и его состояние после каждого прохода внешнего цикла):

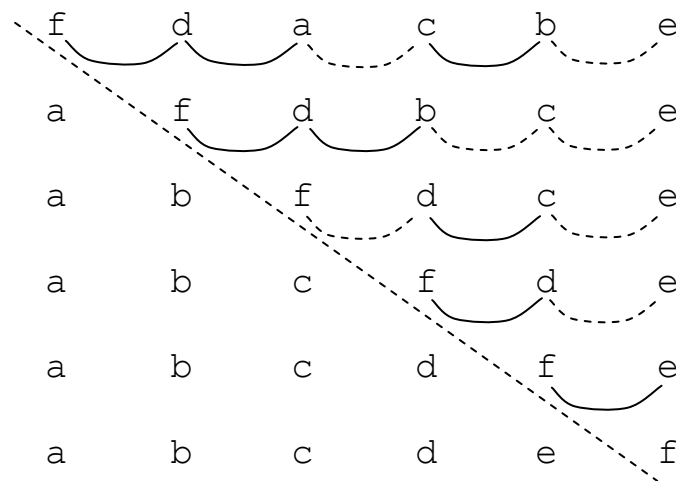


Рис. 13.1 – Изменение массива в процессе пузырьковой сортировки

Рассматриваемый вариант алгоритма всегда выполняет

$$N(N - 1)/2 \quad (13.1)$$

или

$$(N^2 - N)/2 \quad (13.2)$$

операций **сравнения**, так как внешний цикл выполняется $N - 1$ раз, а внутренний $N/2$. Это выражение соответствует площади прямоугольного треугольника с катетами N и $N - 1$.

Число **перестановок** зависит от степени предварительной упорядоченности массива: В **лучшем** случае перестановки не выполняются (массив уже отсортирован в нужном направлении и число перестановок равно 0);

В *среднем* случае выполняется

$$N(N - 1)/4 = (N^2 - N)/4 \quad (13.3)$$

перестановок (т.е. число сравнений делится пополам), число операций присваивания оказывается утроенным, т.е.

$$3N(N - 1)/4. \quad (13.4)$$

В *худшем* случае (массив отсортирован в обратном направлении) выполняется

$$N(N - 1)/2 = (N^2 - N)/2 \quad (13.5)$$

перестановок или

$$3N(N - 1)/2 \quad (13.6)$$

операций присваивания.

Из-за наличия в формулах компонента N^2 , пузырьковая сортировка относится к N -квадратичным алгоритмам (см. рисунок 13.2). При работе с большими структурами данных (массивами или связными списками) пузырьковая сортировка неэффективна.

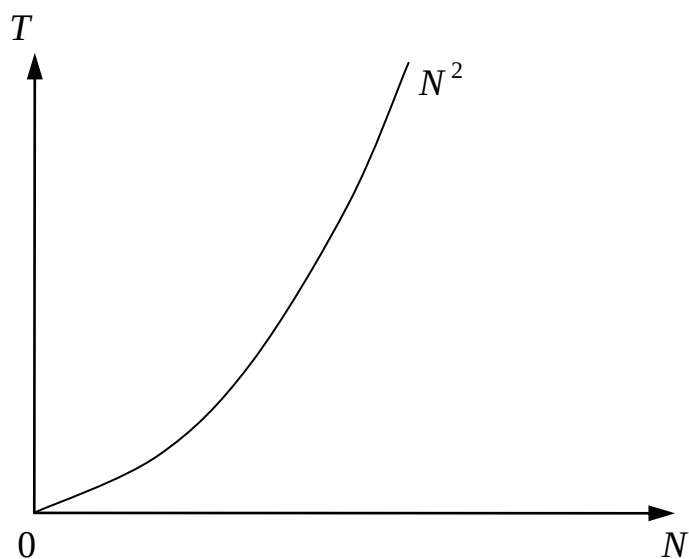


Рис. 13.2 – Зависимость времени работы от размера массива для N -квадратичных алгоритмов

Считается, что пузырьковая сортировка вообще является "учебной", но в реальности она может применяться для упорядочения массивов очень малого размера (3 – 7 элементов) в различных алгоритмах обработки сигналов.

Процедура пузырьковой сортировки на языке Паскаль.

```
var str: array[0..M] of char;
procedure bubble(item: array of char; n: integer);
var
  a, b: integer;
  t: char;
begin
  for a := 1 to n - 1 do
    for b := n - 1 downto a do
      if item[b-1] > item[b] then
        begin
          t := item[b-1]; { Перестановка }
          item[b-1] := item[b];
          item[b] := t
        end
      end;
    end;
  end;
```

Во втором варианте процедуры предпринимается попытка закончить процесс сортировки до завершения внешнего цикла, если все элементы уже заняли свои позиции (т.е. если в очередном проходе внутреннего цикла не было выполнено ни одной операции перестановки) – это соответствует *условию Айверсона* (или *скобке Айверсона*). Этот вариант процедуры длиннее, но позволяет при «удачном» расположении сортируемых данных делать меньше проходов.

```
procedure bubble1(item: array of char; n: integer);
var
  a, ssign, pass: integer;
  t: char;
begin
  ssign := 1;
  pass := 0;
  while ssign <> 0 do
    begin
      ssign := 0;
      pass := pass + 1;
```

```

    for a:=0 to n - pass - 1 do
        if item[a] > item[a + 1] then
            begin
                t := item[a - 1];
                item[a - 1] := item[a];
                item[a] := t;
                ssign := 1
            end
        end
    end
end;

```

Аналогичные функции на языках Си/Си++.

```

void bubble(char* item, int n)
{
    int a, b;
    char buf;
    for (a = 1; a < n; ++a)
        for (b = n - 1; b >= a; --b)
            if (item[b - 1] > item[b])
            {
                buf = item[b - 1];    // Перестановка
                item[b - 1] = item[b];
                item[b] = buf;
            }
}

```

Вариант с условием Айверсона (перестановка не показана):

```

void bubble1(char* item, int n)
{
    int a, ssign = 1, pass = 0;
    char buf;
    while(ssign)
    {
        ssign = 0;
        pass++;
        for (a = 0; a < n - pass; a++)
            if (item[a] > item[a + 1])
            {

```



```

        // Перестановка
        ssign++;
    }
}

```

Условие Айверсона является одной из попыток оптимизации этого алгоритма для повышения его эффективности, но обычно все такие попытки, включая ручную оптимизацию машинных команд, не приводят к заметному повышению скорости работы алгоритма.

Существуют текстовые модификации приведённых процедуры, в которых, например, счётчики обоих циклов меняются на увеличение и т.п.

Тип упорядоченности – по возрастанию или по убыванию – зависит от знака операции сравнения текущего и последующего элементов.

13.3. Сортировка отбором

Другое название – сортировка ***выбором***.

В структуре данных (массиве или списке) ищется наименьший (или наибольший) элемент и меняется местами с первым (или с последним, в зависимости от направления упорядоченности). (Здесь сочетаются два процесса – поиск и обмен.) Затем из оставшихся $N - 1$ элементов ищется наименьший и меняется местами со вторым. Процесс повторяется до тех пор, пока нечего будет переставлять (т.е. пока структура данных не исчерпается).

При реализации алгоритма желательно оптимальным образом совместить процессы поиска минимума и обмена, для достижения максимальной эффективности.

Процедура сортировки простым или прямым выбором на языке Паскаль.

```

procedure select(item: array of char; n: integer);
var
    a, b, c, ssign: integer;
    t: char;

```

```

begin
  for a := 0 to n - 2 do
    begin
      ssign := 0;
      c := a;
      t := item[a];
      for b := a + 1 to n do
        if item[b] < t then
          begin
            c := b;
            t := item[b];
            ssign := 1
          end;
        if ssign <> 0 then
          begin
            item[c] := item[a];
            item[a] := t
          end
        end
      end
    end
  end;
end;

```

Такая же функция на языках Си/Си++.

```

void select_sort(char* item, int n)
{
  int a, b, c;
  char buf;
  int change;
  for(a=0; a < n - 1; ++a)
  {
    buf = item[a]; // Текущий минимум
    c = a;         // Индекс минимума
    change=0;      // Признак обмена
    for (b = a + 1; b < n; ++b)
      if (item[b] < buf)
      {
        buf = item[b]; // Начало обмена
        c = b;         // Новый минимум
        change = 1;    // Признак обмена
      }
  }
}

```

```

    if (change)
    {
        item[c] = item[a]; // Продолжение обмена
        item[a]=buf;
    }
}
}

```

Фактически в этих процедурах во **внешнем** цикле устанавливается нижняя граница интервала анализа и элемент с таким индексом принимается за опорный. Во **внутреннем** цикле каждый из оставшихся элементов сравнивается с опорным и если условие сравнения выполняется, найденный локальный минимум фиксируется в буфере (принимается за новый опорный) и устанавливается в «1» признак обмена. В продолжении **внешнего** цикла происходит обмен, если признак равен «1». Эффективность алгоритма повышается за счёт разнесения операции обмена по разным циклам (по сравнению с пузырьковой сортировкой).

Иллюстрация работы алгоритма показана на рисунке 13.3.

Здесь хорошо виден такой же «треугольный» характер, что и у пузырьковой сортировки.

Этот алгоритм так же является N -квадратичным. Внешний цикл выполняется $N - 1$ раз, внутренний – $N/2$ раз, каждая полная перестановка требует трех операций присваивания.

Число операций сравнения и перестановки (присваивания):

$(N^2 - N)/2$ сравнений;

$N - 1$ присваиваний в **лучшем** случае;

$N^2/4 + 3(N - 1)$ присваиваний в **худшем** случае;

$N(\log_2 N + y)$ присваиваний в **среднем** случае,

где $y \approx 0,577216$ – константа Эйлера.

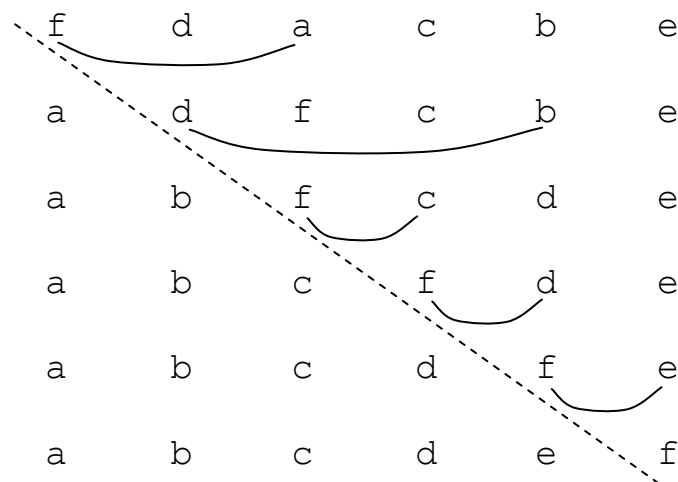


Рис. 13.3 – Обработка массива при сортировке отбором

При одинаковом числе сравнений, сортировка отбором эффективнее пузырьковой в среднем случае за счёт наличия логарифма в выражении для числа перестановок. Лучшая эффективность объясняется в том числе и тем, что в приведённых выше примерах реализации этой сортировки процессы поиска экстремума и обмена данными совмещаются, кроме того разные операции присваивания при обмене данными (метод «трёх вёдер») оказываются разнесёнными в разные части процедуры и выполняются неодинаковое количество раз.

К *модификациям* этого алгоритма относится *пирамидальная* сортировка. Существует также двунаправленный вариант сортировки отбором, в котором на каждом проходе отыскиваются и устанавливаются на свои места и минимальное, и максимальное значения.

13.4. Сортировка вставками

На первом шаге упорядочиваются один относительно другого два первых элемента структуры данных (массива или списка). Затем в упорядочивании участвуют три первых элемента (третий элемент вставляется в соответствующую позицию относительно первых двух элементов), потом – четыре и т. д., пока вся структура данных не будет упорядочена.

Ниже приведен пример алгоритма сортировки *простыми вставками* (просеиванием или погружением).

Процедура сортировки вставками на языке Паскаль.

```
procedure insert(item: array of char; n: integer);
var
  a, b: integer;
  t: char;
begin
  for a:=1 to n - 1 do
    begin
      t := item[a];
      b := a - 1;
      while (b >= 0) and (t < item[b]) do
        begin
          item[b+1] := item[b];
          b := b-1;
        end;
      item[b + 1] := t;
    end;
  end;
```

Процедура сортировки вставками на языках Си/Си++.

```
void insert_sort(char* item, int n)
{
  int a, b;
  char buf;
  for(a=1; a<n; ++a)
  {
    buf = item[a];
    for(b = a-1; b >= 0 && buf < item[b]; --b)
      item[b + 1] = item[b];
    item[b + 1] = buf;
  }
}
```

В отличие от предыдущих алгоритмов, число сравнений зависит от исходной упорядоченности массива. Если массив уже отсортирован в требуемом порядке, то за всё время работы процедуры выполняется $N - 1$ операций сравнения, если не отсорти-

рован, то $(N^2 - N)/2$ сравнений. Количество операций присваивания при перестановках определяется следующим образом [22]:

в лучшем случае – $2(N - 1)$;

в худшем случае оценивается в общем как $(N^2 + 2N)/2$.

Для среднего случая точную оценку дать достаточно сложно, но в общем виде время работы действительно оказывается средним между лучшим и худшим случаями.

В исходном массиве данный алгоритм произведет перестановки, показанные на рисунке 13.4.

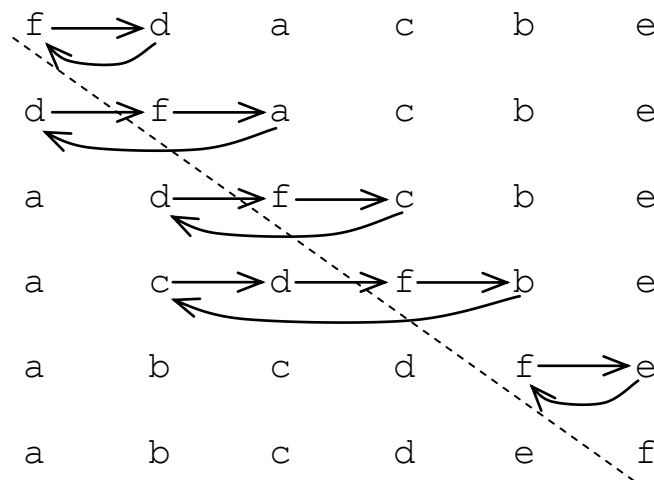


Рис. 13.4 – Обработка массива при сортировке вставками

Этот алгоритм в среднем лишь немного лучше предыдущих (также за счёт разнесения операций присваивания, выполняющих обмен, по разным циклам), но обладает несколькими преимуществами:

- его поведение естественно: если массив уже отсортирован в нужном порядке, алгоритм проводит минимальное количество вычислений, и максимальное, если массив отсортирован в порядке, обратном требуемому. Происходит быстрая обработка почти упорядоченных массивов;

- порядок следования одинаковых ключей не изменяется. Если список отсортирован по двум ключам, то после сортировки вставками он остается отсортированным по обоим ключам;

- одна из самых коротких процедур среди основных алгоритмов сортировки.

Недостаток: при каждой вставке производится сдвиг массива, следовательно даже при относительно малом числе сравнений число перестановок может быть довольно значительным.

Существуют варианты этого алгоритма: бинарные вставки, двухпутевые вставки.

Сортировка вставками хорошо подходит для связных списков, т.к. в этом случае не требуется перенос данных между различными элементами, в отличие от массива.

13.5 Алгоритм Шелла

Разработан Дональдом Л. Шеллом в 1959 году.

Этот алгоритм классифицируется как сортировка ***вставками с убывающим шагом***.

В первом проходе попарно переставляются элементы, находящиеся друг от друга на некотором расстоянии, например, 9 позиций. Во втором проходе переставляются элементы, отстоящие на меньшее число позиций, например, 5. И т. д. На последнем шаге сортируются соседние элементы. На ранних этапах изучения алгоритма его исследователи отмечали: «Непонятно, как алгоритм работает».

Принцип убывающих шагов может быть применён для различных базовых сортировок, в том числе для ***пузырьковой*** сортировки. В результате такой модификации получается сортировка ***расчёской***.

Эффективность алгоритма заключается в том, что на каждом из промежуточных шагов сортируется либо небольшое число элементов, либо уже достаточно хорошо упорядоченные наборы элементов. Упорядоченность массива возрастает после каждого прохода.

Поведение алгоритма Шелла до конца не исследовано до сих пор, в частности, не найдено универсальное решение проблемы выбора последовательности изменения шага. Исследование алгоритма привело к формулированию нескольких теорем.

Эффективность алгоритма проявляется даже в случае всего двух проходов с шагами h и 1. Время работы в этом случае зависит от размера массива как

$$t \approx \frac{2N^2}{h} + \sqrt{\pi N^3 h} . \quad (13.7)$$

А лучшее значение шага h оказывается равным

$$h \approx \sqrt[3]{\frac{16N}{\pi}} \approx 1,72\sqrt[3]{N} , \quad (13.8)$$

следствием этого является зависимость времени работы от размера массива, которую можно выразить как $N^{1,6667}$ или $N^{5/3}$.

При большем числе проходов эффективность повышается. Тем не менее, в процессе изучения алгоритма был обнаружен порог эффективности, который долго не удавалось преодолеть (барьер $N^{1,5}$). Этот барьер можно преодолеть подбором шагов.

Изменение массива при сортировке по алгоритму Шелла будет имеет вид, показанный на рисунке 13.5 (приведены исходное состояние массива, результат каждого прохода внешнего цикла и окончательное состояние).

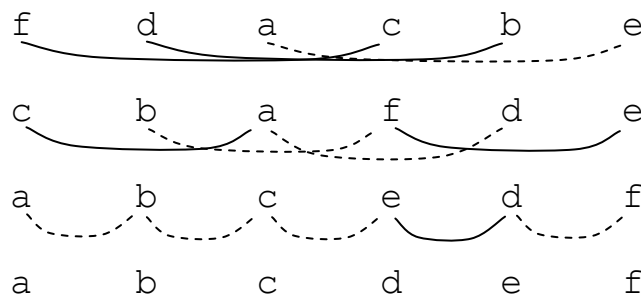


Рис. 13.5 – Обработка массива при сортировке Шелла

В ходе изучения алгоритма исследовалась зависимость среднего числа перестановок от размера массивов при N от 100 до 60000 для нескольких типов последовательностей шагов, для которых были получены соответствующие зависимости времени работ от размера массива:

$$2^{k+1}, \dots, 9, 5, 3, 1 \Rightarrow 1,09N^{1,27}$$

$$2^k - 1, \dots, 15, 7, 3, 1 \Rightarrow 1,22N^{1,26}$$

$$(2^k - (-1)^k)/3, \dots, 11, 5, 3, 1 \Rightarrow 1,12N^{1,28}$$

$$(3^k - 1)/2, \dots, 40, 13, 4, 1 \Rightarrow 1,66N^{1,25}$$

Качественное сравнение времени работы для N -квадратичных сортировок и метода Шелла приведено на рисунке 13.6.

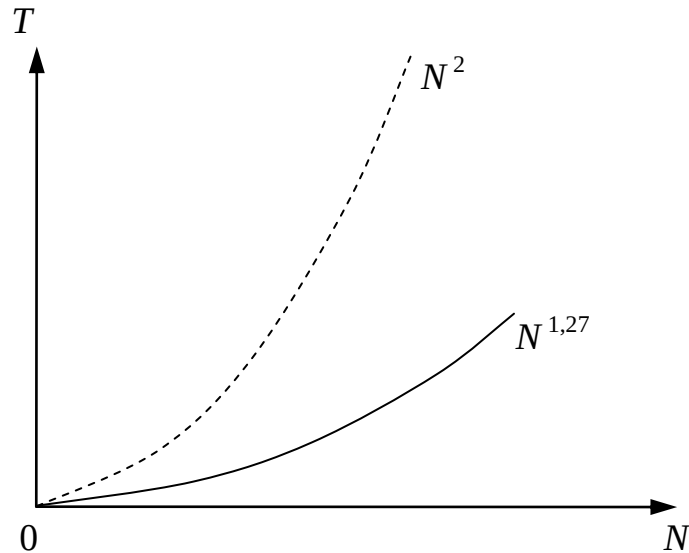


Рис. 13.6 – Сравнение зависимости времени работы от размера массива для алгоритма Шелла и N -квадратичных алгоритмов

Общую формулу можно выразить как

$$0,3N(\ln N)^2 - 1,35N(\ln N), \quad (13.9)$$

или, упростив, как

$$\alpha N(\ln N)^2 + \beta N \ln N \quad (13.10)$$

или αN^β .

Последний шаг обязательно должен быть единичным.

Распространённой (хотя и далеко не самой эффективной) является последовательность шагов 9, 5, 3, 2, 1.

Кроме указанных формул могут использоваться числа Фибоначчи.

Рекомендуется избегать последовательности шагов в виде степеней двойки (... , 16, 8, 4, 2, 1), так как доказано, что это снижает эффективность алгоритма (хотя сортировка выполняется и в этом случае).

Один из рекомендуемых вариантов выбора последовательности шагов: принять шаги последнего и каждого предыдущего этапов $h_1 = 1$, $h_{s+1} = 3h_s + 1$, и остановиться на некотором шаге h_t , когда $h_{t+2} \geq N$. В итоге получается последовательность шагов ..., 121, 40, 13, 4, 1 (такая же, как в случае $(3^k - 1)/2$), которая является одной из самых эффективных. Возможны и другие, эвристически подбираемые последовательности шагов, ещё более эффективные

(например, 120, 40, 12, 4, 1). Разумеется, большие шаги могут применяться только для массивов большого размера.

Возможен также расчёт шага первого этапа, исходя из размера массива, и уменьшение шага вдвое на каждом следующем этапе.

Процедура сортировки методом Шелла на языке Паскаль.

```
procedure shell(item: array of char; n: integer);
var
  i, j, k, h: Integer;
  x: char;
  a: array[0..4] of integer;
begin
  a[0] := 9; a[1] := 5;
  a[2] := 3; a[3] := 2;
  a[4] := 1;
  for k := 0 to 4 do
    begin
      h := a[k];
      for i := h to n-1 do
        begin
          x := item[i];
          j := i-h;
          while (x < item[j]) and ( j >= 1) do
            begin
              item[j + h] := item[j];
              j := j-h;
            end;
            item[j + h] := x;
          end
        end
      end
    end;
end;
```

Аналогичная функция на Си++.

```
const int ST = 5;
void Shell_sort(char* item, int n)
{
  int step[ST] = {120, 40, 12, 4, 1}; // Массив убывающих шагов
```

```

int i, j, k, h;
char buf;
for ( k = 0; k < ST; ++k)
{
    h = step[k];
    for (i = h; i < n; ++i)
    {
        buf = item[i];
        for (j = i - h; buf < item[j] && j >= 0;
j -= h)
            item[j + h] = item[j];
        item[j + h] = buf;
    }
}

```

Проверка условия $j \geq 1$ (для Паскаля) или $j \geq 0$ (для Си++) предотвращает выход за пределы массива. Считается, что такие дополнительные проверки слегка снижают производительность алгоритма.

13.6. Алгоритм быстрой сортировки

Классификационное название этого алгоритма – **обменная сортировка с разделением**.

Разработан Чарльзом Э. Р. Хоаром в 1962 г.

Один из лучших универсальных алгоритмов (одновременно не очень сложных), разработанных к настоящему времени. Относится к группе алгоритмов **обменной** сортировки (как и **пузырьковая** сортировка).

Сортируемый массив разбивается на два подмассива, для чего сначала выбирается пограничное или опорное значение – **компаранд** (т.е. значение, с которым будут **сравниваться** другие элементы массива). Все элементы, **большие** компаранда, переносятся в один подмассив, а **меньшие** – в другой. Весь процесс **повторяется** для **каждого** подмассива до тех пор, пока весь массив не будет упорядочен.

Процесс сортировки носит рекурсивный характер.

Перенос в подмассивы может происходить следующим образом:

Пусть имеются два индекса – i и j , причем сначала $i = 0$, $j = N - 1$.

Сравниваются элементы $k[i]$ и $k[j]$, если обмен не требуется, то j уменьшается на единицу и сравнения повторяются, j с каждым шагом уменьшается на 1.

После первого обмена i увеличивается на единицу и сравнения повторяются с увеличением i на 1, пока не произойдет следующий обмен, после которого опять начинает уменьшаться j . И так до тех пор, пока i и j не станут равны друг другу.

Представляет интерес вопрос о выборе компаранда. Существуют варианты:

- **случайный** выбор – иногда бывает наилучшим;
- выборка небольшого числа элементов из подмассива и использования в качестве компаранда медианы этой выборки. Широко распространенным и эффективным является метод выборки трех элементов и взятие среднего из них;
- один из **экстремумов** – наихудший вариант, хотя алгоритм работает и в этом случае;
- элемент, находящийся в середине каждого из подмассивов.

Процедура быстрой сортировки (метод Хоара) на языке Паскаль с компарандом в середине каждого текущего подмассива.

```
procedure quicksort(item: array of char; left,
right: integer);
var
  i, j: Integer;
  x, y: char;
begin
  i := left;
  j := right;
  x := item[int((left + right) / 2)];
  repeat
    while (item[i] < x) and (i < right) do
      i := i + 1;
    while (x < item[j]) and (j > left) do
```

```

        j := j - 1;
    if i <= j then
    begin
        y := item[i];
        item[i] := item[j];
        item[j] := y;
        i := i + 1; j := j - 1;
    end;
until i < j;
if left < j then
    quick(item, left, j);
if i < right then
    quick(item, i, right);
end;

```

Аналогичная функция на языках Си/Си++.

```

void quick_sort(char* item,int left,int right)
{
    int i, j;
    char comp, buf;
    i = left; j = right;
    comp = item[(left + right)/2]; // Компаранд
    do {
        while (item[i] < comp && i < right)
            i++;
        while (comp < item[j] && j > left)
            j--;
        if (i<=j)
        {
            buf = item[i]; // Обмен
            item[i] = item[j];
            item[j] = buf;
            i++;
            j--;
        }
    } while(i <= j);
    if (left < j)
        quick_sort(item, left, j); // Рекурсивный

```

ВЫЗОВ

```

    if (i < right)
        quick_sort(item, i, right);
}

```

Изменение массива при сортировке по алгоритму Хоара будет иметь вид, показанный на рисунке 13.7 (компаранды подчеркнуты, обрабатываемые подмассивы обведены).

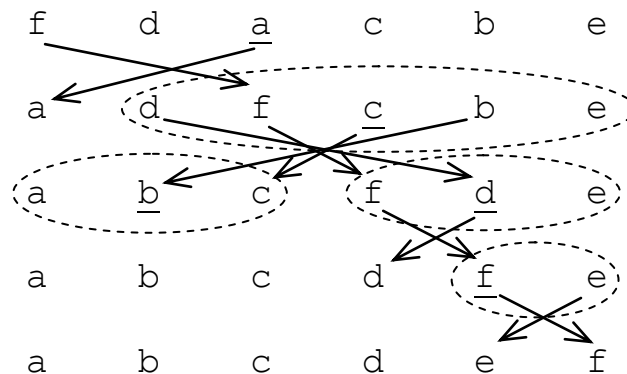


Рис. 13.7 – Обработка массива при быстрой сортировке

Для оптимальной сортировки в качестве компаранда следует выбрать элемент, расположенный точно в середине **диапазона** значений текущего подмассива. Однако для большинства наборов данных поиск такого элемента может вылиться в самостоятельную сортировку.

Характеристики: среднее число **сравнений** $N \log_2 N$, среднее число **операций присваивания** при перестановках $(1/6)N \log_2 N$ [22].

У этого алгоритма есть одна отрицательная особенность: если для каждого текущего подмассива компарандом является текущий экстремум (как на первом и последнем этапах сортировки, показанной на рисунке 13.7), то алгоритм становится N -квадратичным. Однако считается, что вероятность такого события обычно невысока. Возможность вырождения быстрой сортировки в N -квадратичную является недостатком этого алгоритма. От такого недостатка свободны сортировки **слиянием** и **пирамидальная**, также обладающие эффективностью $O(N \log_2 N)$, но существенно более сложные, чем алгоритм Хоара.

13.7. Параллельная сортировка Бэтчера

Обменная сортировка со слиянием. Разработана К. Э. Бэтчером в 1964 г.

Алгоритм напоминает **алгоритм Шелла** – также сравниваются пары несоседних ключей, но пары подбираются по-другому. Используется последовательность шагов 8, 4, 2, 1, но сравниваемые пары не перекрываются (т.е. в двух сравниваемых соседних парах нет общего элемента, как в алгоритме Шелла – 1-й и 9-й, 9-й и 17-й, 17-й и 26-й и т.д.). Далее происходит слияние пар отсортированных подмассивов.

Процедура параллельной сортировки Бэтчера на языке Паскаль.

```
procedure batcher (item: array of char;
  n: Integer);
var
  f, h, i, q, r, z: Integer;
  buf: char;
begin
  (* f:=Log2n; h:=2^(f-1); *) (* Библиотечные
функции логарифма и возведения в степень *)
  while h > 0 do
    begin
      r := 0;
      z := h;
      (* q:=2^(f-1); *)
      while q >= h do
        begin
          for i:=0 to n-z-1 do
            if (i and h) = r then
              z := q - h; (* сравнение и обмен *)
              q := q / 2;
              r := h
            end;
          h := h div 2
        end
      end;
    end;
```

Последовательность сравнений и перестановок, а также состояния переменных, показаны на рисунке 13.8.

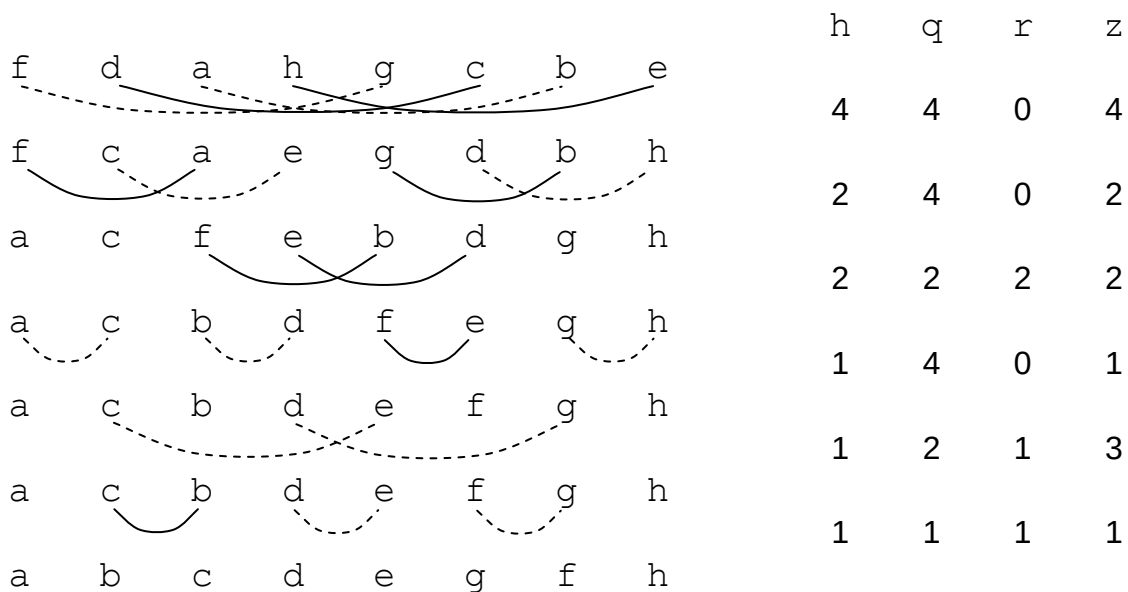


Рис. 13.8 – Сортировка массива алгоритмом Бэтчера

Выбирается начальный интервал сравнения пар h из условия $h = 2^{f-1}$, где $f = \log_2 N$ – наименьшее целое число, такое, что $2^f \geq N$. Перед первым проходом устанавливаются вспомогательные переменные

$$q = 2^{f-1}, r = 0, z = h.$$

Выполняется сравнение и обмен элементов для всех i , таких, что $0 \leq i < (N - z)$ и $i \& h = r$ (операция И над i и h), т.е. при необходимости меняются местами элементы с индексами $i + 1$ и $i + z + 1$, $i + k$ и $i + z + k$ и т.п.

К этому моменту z – нечетно кратно h (т.е. z / h – нечетное число), а h – степень двойки, следовательно $(i \& h) \neq ((i + z) \& h)$, значит, сравнение и обмен можно выполнять при всех нужных значениях i в любом порядке или даже одновременно. В связи с этим сортировка и названа **параллельной**.

Затем изменяются значения вспомогательных переменных $z = q - h$, $q = q / 2$, $r = h$ и процесс продолжается, пока $q \geq h$.

К этому моменту исходный массив будет упорядочен с шагом h .

Затем h уменьшается в два раза ($h = h / 2$) (с учетом целого типа) и весь процесс продолжается, пока $h > 0$.

Значительное количество вспомогательных операций, необходимых для управления последовательностью сравнений, снижает эффективность алгоритма.

Но все сравнения и обмены можно выполнять одновременно (если существует такая возможность, например на ЭВМ с распараллеливанием вычислений). В этом случае сортировка выполняется за $(\log_2 N (\log_2 N + 1)) / 2$ шагов.

Вопросы и задания для самоконтроля

- 13.1. Что такое «сортировка»?
- 13.2. Сколько существует групп алгоритмов сортировки?
- 13.3. Сколько существует алгоритмов сортировки?
- 13.4. По каким признакам характеризуются алгоритмы сортировки?
- 13.5. Что нужно учитывать при выборе алгоритма сортировки?
- 13.6. Какой алгоритм сортировки считается самым простым?
- 13.7. Какой алгоритм сортировки считается самым эффективным?
- 13.8. Что означает понятие «скорость сортировки»?
- 13.9. В чем заключается метод пузырьковой сортировки?
- 13.10. Модифицируйте рассмотренную в параграфе 13.2 функцию пузырьковой сортировки так, чтобы можно было контролировать состояние массива на каждом этапе работы алгоритма.
- 13.11. Предложите функцию пузырьковой сортировки для односвязного списка.
- 13.12. В чем заключается метод сортировки отбором?
- 13.13. Модифицируйте рассмотренную в параграфе 13.3 функцию сортировки отбором так, чтобы можно было контролировать состояние массива на каждом этапе работы алгоритма.
- 13.14. Изобразите диаграмму изменения состояния массива, аналогичную рисунку 13.1, для пузырьковой сортировки при худшем случае.

13.15. Изобразите диаграмму изменения состояния массива, аналогичную рисунку 13.1, для пузырьковой сортировки при лучшем случае.

13.16. Предложите функцию сортировки отбором для односвязного списка.

13.17. В чем заключается метод сортировки вставками?

13.18. Модифицируйте рассмотренную в параграфе 13.4 функцию сортировки вставками так, чтобы можно было контролировать состояние массива на каждом этапе работы алгоритма.

13.19. Модифицируйте функцию сортировки вставками так, чтобы можно было контролировать количество операций сравнения и присваивания.

13.20. Как будет изменяться при сортировке вставками односвязный линейный список?

13.21. Предложите функцию сортировки вставками для односвязного списка.

13.22. Изобразите диаграмму изменения состояния массива, аналогичную рисунку 13.4, для сортировки вставками при худшем случае.

13.23. В чем заключается метод сортировки Шелла?

13.24. Модифицируйте рассмотренную в параграфе 13.5 функцию сортировки методом Шелла так, чтобы можно было контролировать состояние массива на каждом этапе работы алгоритма.

13.25. Можно ли применить метод Шелла для сортировки связного списка?

13.26. Оправданно ли с точки зрения эффективности применение сортировки Шелла для связного списка?

13.27. В чем заключается метод быстрой сортировки?

13.28. Модифицируйте рассмотренную в параграфе 13.6 функцию быстрой сортировки так, чтобы можно было контролировать состояние массива на каждом этапе работы алгоритма.

13.29. В чем заключается метод сортировки Бэтчера?

13.30. Как зависит скорость сортировки от размера структуры данных для разных алгоритмов?

13.31. Почему метод Бэтчера называется параллельным?

13.32. Каковы критерии выбора алгоритма сортировки?